HelixMemory

HelixMemory

Tagline: One memory brain for AI agents — four best-in-class engines, fused.

Summary: HelixMemory is a Go SDK that unifies four leading memory systems (Mem0, Cognee, Letta, Graphiti) into a single cognitive memory engine that searches them in parallel and fuses the results. It gives AI applications one durable, deduplicated, re-ranked memory layer instead of four disconnected ones.

Short description (~40 words): HelixMemory is a Go SDK that fuses Mem0, Cognee, Letta and Graphiti into one unified cognitive memory engine for AI applications. It routes writes intelligently, searches every backend in parallel, and fuses results through a three-stage collect-dedupe-rerank pipeline.

Long description (150-250 words): HelixMemory is a unified cognitive memory engine for AI applications, delivered as a Go SDK (module `digital.vasic.helixmemory`, Go 1.25+). Rather than re-implementing memory from scratch, it orchestrates four best-in-class systems and lets each do what it does best: Mem0 for dynamic fact extraction and preference management, Cognee for semantic knowledge graphs via ECL pipelines, Letta for stateful agent runtime with editable memory blocks and sleep-time compute, and Graphiti for a bi-temporal knowledge graph.

A fusion engine ties them together. Writes are classified by content and routed to the optimal backend; reads fan out across all backends in parallel and pass through a three-stage fusion pipeline — collection, deduplication, and cross-source re-ranking — so the caller sees one clean, ranked memory result. Circuit breakers provide graceful degradation: if one backend is down, the rest keep serving. A drop-in `MemoryStore` interface makes it a direct replacement for a plain memory provider, and Prometheus metrics expose full observability.

HelixMemory was built as the memory layer for HelixAgent, the wider Helix AI ensemble. It follows the family’s anti-bluff testing discipline: an in-process challenge runner exercises real

production code (routing, fusion, translator, circuit breaker) and a paired-mutation wrapper proves the tests actually fail when the invariants are broken.

Why we built it: AI agents need long-lived, high-quality memory, but the ecosystem is fragmented — each memory project (Mem0, Cognee, Letta, Graphiti) is strong at one thing and weak at others. HelixMemory was built to give HelixAgent a single memory surface that combines their strengths without forcing a lock-in to any one of them.

Why it's a game-changer: It turns four competing memory systems into complementary backends behind one interface. Teams get fact extraction, semantic graphs, stateful agent memory, and temporal reasoning at once — with automatic deduplication and re-ranking — instead of choosing one and living with its blind spots.

What's innovative: - Multi-backend **fusion** (collect → dedupe → cross-source re-rank) rather than a single store. - **Intelligent routing** that classifies memory by content and sends it to the best-suited engine. - **Graceful degradation** via per-backend circuit breakers. - **Sleep-time compute** consolidation during idle periods. - Anti-bluff verification: challenge runner over real code plus a paired-mutation proof that the gate is not a tautology.

Biggest technical challenges + how solved: - *Reconciling heterogeneous backends into one result set* — solved with a typed fusion engine that deduplicates overlapping results and re-ranks across sources, with the fused-count invariant asserted in tests. - *Reliability when a backend fails* — solved with circuit breakers (closed → open after threshold → half-open after timeout) so the system keeps serving from healthy backends. - *Proving the memory logic actually works, not just compiles* — solved with an in-process challenge runner over production code and a mutation wrapper that must fail when an invariant is flipped.

Tech stack: - **Go (1.25+)** — single SDK/runtime; chosen for concurrency (parallel backend fan-out) and a clean interface surface (`MemoryStore`). - **Mem0** — dynamic fact extraction + preference management backend. - **Cognee** — semantic knowledge-graph backend built on ECL pipelines. - **Letta** — stateful agent-runtime backend with editable memory blocks and sleep-time compute. - **Graphiti** — temporal (bi-temporal) knowledge-graph backend. - **PostgreSQL + Neo4j + Redis** — used by the real backend stack for integration testing (`make infra-start`). - **Prometheus** — metrics/observability across the fusion pipeline. - **i18n translator seam** — namespaced (`helixmemory_`) string surface for any future user-facing layer.

Public links: - GitHub: https://github.com/HelixDevelopment/memory (public; note: the display name “HelixMemory” maps to the repo `memory`) - License: no LICENSE detected via the GitHub API — UNVERIFIED / not declared.

Suggested diagrams/illustrations (OpenDesign): 1. Fusion architecture diagram: four backend boxes (Mem0 / Cognee / Letta / Graphiti) feeding a central router and a three-stage fusion pipeline into one unified result. 2. Write-vs-read flow: a memory being classified and routed on write; a query fanning out and re-ranking on read. 3. Circuit-breaker state machine (closed → open → half-open) illustrating graceful degradation.

Site relevance: both (vasic.digital as a developer/AI-infrastructure product; milosvasic.ru as a flagship AI-engineering highlight).

Priority tier: Helix-primary.

Source provenance: - `gh api repos/HelixDevelopment/memory` (metadata: public, Go, no license, created 2026-02-24, pushed 2026-06-24, 1 star). - `gh api repos/HelixDevelopment/memory/readme` (features, fusion stages, backends, anti-bluff round-274 runner). - `/Volumes/T7/Projects/vasic/_analysis/github-helix-others.{md,json}` (description, product-family relationships). - UNVERIFIED: license not declared; accuracy figures cited in README (e.g., Mem0 “26%+ over baseline”, Cognee “38+ connectors”) are upstream vendors’ claims, not HelixMemory measurements — omitted from marketing copy above.